

ARTI 502

Deep Learning: 1st Assignment

Training-Testing Neural Net with MNIST Dataset using Pytorch

Year 2026 Term 1

Grade: 7.5/7.5 (according to the rubric grading)

With a final accuracy of:98.44% after tuning

Group #6

Group Members:

Member	ID
Hashim Ali Aljarrash	2230000215
Ahmad Muhanna	2230006180
Mohammed Alammari	2230003182
Ali Alshehri	2230002515
Osama Al Haddaf	2230001201

Overview

In this assignment, you will perform Training Neural Net on the MNIST Dataset dataset using Pytorch. The goal is to explore the Basic Neural Network, understand its structure, and identify important issues such as architecture, learning rate, momentum and other parameters. You will also analyze relationships between number of layers and number of neurons for each layers to the overall Neural Network performance through their tuning hyperparameters.

Objective

Explore the dataset using PyTorch to understand:

- Simple Neural Networks structure
- How to do Training-Testing Neural Networks
- How to evaluate the performance
- How to determine learning rate, momentum and model hyperparameters

Tasks (with PyTorch)

Task 1: Install PyTorch

```

import random
import torch
import torchvision
#-----
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
#-----
import matplotlib.pyplot as plt
#-----
import torch.nn as nn
import torch.nn.functional as F
#-----
import torch.optim as optim
from torch.utils.data import random_split

SEED = 42

random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
g = torch.Generator()
g.manual_seed(SEED)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

```

[6] ✓ 0.0s

The device working on CPU or GPU:

```

# working with cpu currently because of amd GPU support inconsistency
print("PyTorch version: {}".format(torch.__version__))
print("Torchvision version: {}".format(torchvision.__version__))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device: {}".format(device))

```

[7] ✓ 0.0s

```

... PyTorch version: 2.14.0+cpu
Torchvision version: 0.29.0+cpu
Using device: cpu

```



Task 2: Download and Load the Dataset

Task 2 loading the dataset

```
transform = transforms.Compose([transforms.ToTensor()])

train_dataset = torchvision.datasets.MNIST(
    root="./data", train=True, download=True, transform=transform
)
test_dataset = torchvision.datasets.MNIST(
    root="./data", train=False, download=True, transform=transform
)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True, generator=g)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

print("Train dataset size: {}".format(len(train_dataset)))
print("Test dataset size: {}".format(len(test_dataset)))
```

✓ 0.0s

Train dataset size: 60000
Test dataset size: 10000

Task 3: Inspect the Dataset

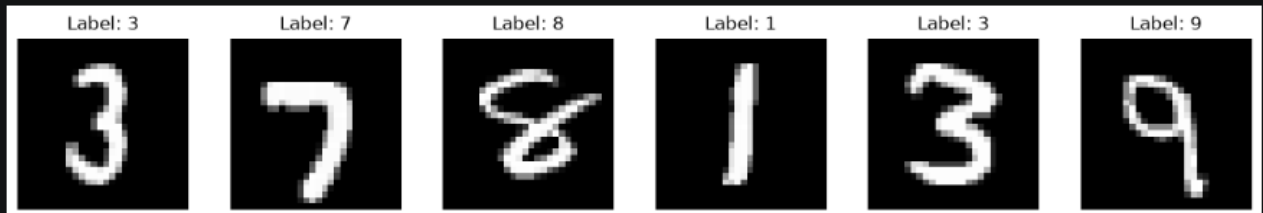
```
images, labels = next(iter(train_loader))
print("Images tensor shape: {}".format(images.shape))
print("Labels tensor shape: {}".format(labels.shape))

fig, axes = plt.subplots(1, 6, figsize=(12, 2))
for i in range(6):
    axes[i].imshow(images[i][0], cmap="gray")
    axes[i].set_title("Label: {}".format(labels[i].item()))
    axes[i].axis("off")
plt.tight_layout()
plt.show()
```

✓ 0.1s

Python

Images tensor shape: torch.Size([32, 1, 28, 28])
Labels tensor shape: torch.Size([32])



Task 4: Creating A Simple Network

```
class SimpleNeuralNetwork(nn.Module):
    def __init__(self, input_size=784, hidden_size=128, num_classes=10):
        super(SimpleNeuralNetwork, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        x = torch.flatten(x, start_dim=1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

```
net = SimpleNeuralNetwork()
print(net)

sample, _ = next(iter(train_loader))
flattened = torch.flatten(sample, start_dim=1)
output = net(sample)
print("Input shape: {}".format(sample.shape))
print("Flattened shape: {}".format(flattened.shape))
print("Output shape: {}".format(output.shape))
```

✓ 0.0s

```
SimpleNeuralNetwork(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)
Input shape: torch.Size([32, 1, 28, 28])
Flattened shape: torch.Size([32, 784])
Output shape: torch.Size([32, 10])
```

Task 5: Training Function

Task 5 Training function

```
def train(net, dataloader, epochs=5, lr=0.01, momentum=0.9, device="cpu", print_every=200, val_loader=None):
    net.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(net.parameters(), lr=lr, momentum=momentum)

    losses = []
    accuracies = []
    val_accuracies = []

    for epoch in range(epochs):
        net.train()
        running_loss = 0.0
        correct = 0
        total = 0

        for batch_idx, (images, labels) in enumerate(dataloader):
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = net(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            losses.append(loss.item())

            _, predicted = torch.max(outputs, 1)
            correct += (predicted == labels).sum().item()
            total += labels.size(0)

            if batch_idx % print_every == 0:
                print("Epoch {} | Batch {} | Loss: {:.4f}".format(epoch + 1, batch_idx, loss.item()))

        epoch_acc = correct / total
        accuracies.append(epoch_acc)

        log_line = "Epoch {} finished | Avg Loss: {:.4f} | Train Acc: {:.4f}".format(
            epoch + 1, running_loss / len(dataloader), epoch_acc)

        if val_loader is not None:
            val_acc = test(net, val_loader, device=device)
            val_accuracies.append(val_acc)
            log_line += " | Val Acc: {:.4f}".format(val_acc)

        print(log_line)

    if val_loader is not None:
        return losses, accuracies, val_accuracies
    return losses, accuracies
```

[11] ✓ 0.0s

Task 6: Training Network

Task 6 training the network

```
net = SimpleNeuralNetwork(hidden_size=128)
losses, accuracies, val_accuracies = train(net, train_loader, epochs=15, lr=0.01, momentum=0.9, device=device, val_loader=test_loader)
```

✓ 1m 18.7s

```
Epoch 1 | Batch 0 | Loss: 2.3160
Epoch 1 | Batch 200 | Loss: 0.4143
Epoch 1 | Batch 400 | Loss: 0.6063
Epoch 1 | Batch 600 | Loss: 0.1356
Epoch 1 | Batch 800 | Loss: 0.1067
Epoch 1 | Batch 1000 | Loss: 0.5376
Epoch 1 | Batch 1200 | Loss: 0.2431
Epoch 1 | Batch 1400 | Loss: 0.3609
Epoch 1 | Batch 1600 | Loss: 0.0342
Epoch 1 | Batch 1800 | Loss: 0.2704
Test Accuracy: 0.9449
Epoch 1 finished | Avg Loss: 0.3560 | Train Acc: 0.9006 | Val Acc: 0.9449
Epoch 2 | Batch 0 | Loss: 0.1369
Epoch 2 | Batch 200 | Loss: 0.1284
Epoch 2 | Batch 400 | Loss: 0.3126
Epoch 2 | Batch 600 | Loss: 0.0756
Epoch 2 | Batch 800 | Loss: 0.0650
Epoch 2 | Batch 1000 | Loss: 0.1708
Epoch 2 | Batch 1200 | Loss: 0.1288
Epoch 2 | Batch 1400 | Loss: 0.0362
Epoch 2 | Batch 1600 | Loss: 0.1769
Epoch 2 | Batch 1800 | Loss: 0.0309
Test Accuracy: 0.9617
Epoch 2 finished | Avg Loss: 0.1623 | Train Acc: 0.9537 | Val Acc: 0.9617
Epoch 3 | Batch 0 | Loss: 0.0670
...
Epoch 15 | Batch 1600 | Loss: 0.0024
Epoch 15 | Batch 1800 | Loss: 0.0415
Test Accuracy: 0.9782
Epoch 15 finished | Avg Loss: 0.0173 | Train Acc: 0.9965 | Val Acc: 0.9782
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

```

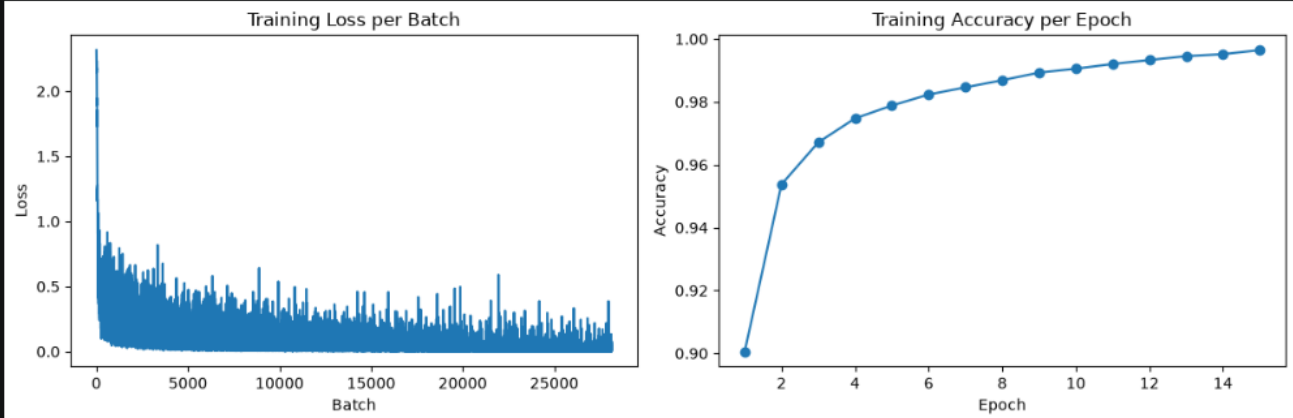
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(losses)
axes[0].set_title("Training Loss per Batch")
axes[0].set_xlabel("Batch")
axes[0].set_ylabel("Loss")

axes[1].plot(range(1, len(accuracies) + 1), accuracies, marker="o")
axes[1].set_title("Training Accuracy per Epoch")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy")

plt.tight_layout()
plt.show()

```



Task 7: Testing Network

Task 7 testing the network

```
def test(net, dataloader, device="cpu"):
    net.to(device)
    net.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in dataloader:
            images, labels = images.to(device), labels.to(device)
            outputs = net(images)
            _, predicted = torch.max(outputs, 1)
            correct += (predicted == labels).sum().item()
            total += labels.size(0)
    accuracy = correct / total
    print("Test Accuracy: {:.4f}".format(accuracy))
    return accuracy
```

```
test_accuracy = test(net, test_loader, device=device)
```

✓ 0.6s

Test Accuracy: 0.9795

```
images, labels = next(iter(test_loader))
images, labels = images.to(device), labels.to(device)
outputs = net(images)
_, predicted = torch.max(outputs, 1)

fig, axes = plt.subplots(1, 6, figsize=(12, 2))
for i in range(6):
    axes[i].imshow(images[i][0].cpu(), cmap="gray")
    axes[i].set_title("Pred: {} / True: {}".format(predicted[i].item(), labels[i].item()))
    axes[i].axis("off")
plt.tight_layout()
plt.show()
```

✓ 0.1s

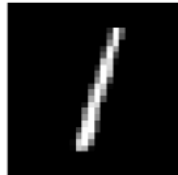
Pred: 7 / True: 7



Pred: 2 / True: 2



Pred: 1 / True: 1



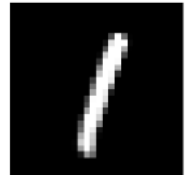
Pred: 0 / True: 0



Pred: 4 / True: 4



Pred: 1 / True: 1



Task 8: Tuning the parameters and model architecture

A. Tuning the learning rate and momentum:

```
# tuning the learning rate and momentum

tune_train_subset, tune_val_subset = random_split(train_dataset, [50000, 10000], generator=g)
tune_train_loader = DataLoader(tune_train_subset, batch_size=32, shuffle=True, generator=g)
tune_val_loader = DataLoader(tune_val_subset, batch_size=32, shuffle=False)
TUNE_EPOCHS = 4
lr_values = [0.001, 0.01, 0.1]
lr_results = []
for lr in lr_values:
    print("Testing lr={}".format(lr))
    torch.manual_seed(SEED)
    net_cfg = SimpleNeuralNetwork(hidden_size=128)
    _, _, val_acc_cfg = train(
        net_cfg, tune_train_loader, epochs=TUNE_EPOCHS, lr=lr, momentum=0.9,
        device=device, print_every=10000, val_loader=tune_val_loader
    )
    lr_results.append({"lr": lr, "momentum": 0.9, "val_acc": val_acc_cfg[-1]})
    print("-" * 50)

best_lr = max(lr_results, key=lambda r: r["val_acc"])["lr"]
print("Best learning rate: {} (momentum fixed at 0.9)".format(best_lr))

momentum_values = [0.0, 0.5, 0.9, 0.99]
momentum_results = []
for m in momentum_values:
    print("Testing momentum={}".format(m))
    torch.manual_seed(SEED)
    net_cfg = SimpleNeuralNetwork(hidden_size=128)
    _, _, val_acc_cfg = train(
        net_cfg, tune_train_loader, epochs=TUNE_EPOCHS, lr=best_lr, momentum=m,
        device=device, print_every=10000, val_loader=tune_val_loader
    )
    momentum_results.append({"lr": best_lr, "momentum": m, "val_acc": val_acc_cfg[-1]})
    print("-" * 50)

best_lr_momentum = max(momentum_results, key=lambda r: r["val_acc"])
print("Best lr/momentum config: lr={}, momentum={}".format(
    best_lr_momentum["lr"], best_lr_momentum["momentum"]))
print("Best val accuracy: {:.4f}".format(best_lr_momentum["val_acc"]))

print("\nLR sweep results:", lr_results)
print("Momentum sweep results:", momentum_results)
```

✓ 2m 2.7s

```
Epoch 1 finished | Avg Loss: 0.3831 | Train Acc: 0.8916 | Val Acc: 0.9357
Epoch 2 | Batch 0 | Loss: 0.0935
Test Accuracy: 0.9524
Epoch 2 finished | Avg Loss: 0.1882 | Train Acc: 0.9485 | Val Acc: 0.9524
Epoch 3 | Batch 0 | Loss: 0.0813
Test Accuracy: 0.9599
Epoch 3 finished | Avg Loss: 0.1259 | Train Acc: 0.9646 | Val Acc: 0.9599
Epoch 4 | Batch 0 | Loss: 0.0570
Test Accuracy: 0.9656
Epoch 4 finished | Avg Loss: 0.0989 | Train Acc: 0.9718 | Val Acc: 0.9656
-----
Testing momentum=0.99
Epoch 1 | Batch 0 | Loss: 2.2981
Test Accuracy: 0.9369
Epoch 1 finished | Avg Loss: 0.3972 | Train Acc: 0.8920 | Val Acc: 0.9369
Epoch 2 | Batch 0 | Loss: 0.0989
Test Accuracy: 0.9462
Epoch 2 finished | Avg Loss: 0.2010 | Train Acc: 0.9426 | Val Acc: 0.9462
Epoch 3 | Batch 0 | Loss: 0.1621
Test Accuracy: 0.9496
Epoch 3 finished | Avg Loss: 0.1722 | Train Acc: 0.9518 | Val Acc: 0.9496
Epoch 4 | Batch 0 | Loss: 0.0132
Test Accuracy: 0.9522
Epoch 4 finished | Avg Loss: 0.1526 | Train Acc: 0.9592 | Val Acc: 0.9522
-----
Best lr/momentum config: lr=0.01, momentum=0.9
Best val accuracy: 0.9656

LR sweep results: [{'lr': 0.001, 'momentum': 0.9, 'val_acc': 0.9119}, {'lr': 0.01, 'momentum': 0.9, 'val_acc': 0.9652}, {'lr': 0.1, 'momentum': 0.9, 'val_acc': 0.9576}]
Momentum sweep results: [{'lr': 0.01, 'momentum': 0.0, 'val_acc': 0.9134}, {'lr': 0.01, 'momentum': 0.5, 'val_acc': 0.9384}, {'lr': 0.01, 'momentum': 0.9, 'val_acc': 0.9656}, {'lr': 0.01, 'momentum': 0.99, 'val_acc': 0.9522}]
```

Best learning rate and momentum configuration: lr=0.01, momentum=0.9 with accuracy of 0.9656

B. tuning the architecture (number of hidden layers and neurons):

```
# tuning the number of layers and neurons
class TuneNet(nn.Module):
    def __init__(self, input_size=784, hidden_sizes=(256, 128), num_classes=10):
        super(TuneNet, self).__init__()
        layers = []
        in_size = input_size
        for h in hidden_sizes:
            layers.append(nn.Linear(in_size, h))
            layers.append(nn.ReLU())
            in_size = h
        layers.append(nn.Linear(in_size, num_classes))
        self.model = nn.Sequential(*layers)

    def forward(self, x):
        x = torch.flatten(x, start_dim=1)
        return self.model(x)

BEST_LR = best_lr_momentum["lr"]
BEST_MOMENTUM = best_lr_momentum["momentum"]

width_values = [64, 128, 256, 512]
width_results = []
for w in width_values:
    print("Testing width={}".format(w))
    torch.manual_seed(SEED)
    net_cfg = TuneNet(hidden_sizes=(w,))
    _, _, val_acc_cfg = train(
        net_cfg, tune_train_loader, epochs=TUNE_EPOCHS, lr=BEST_LR, momentum=BEST_MOMENTUM,
        device=device, print_every=10000, val_loader=tune_val_loader
    )
    width_results.append({"hidden_sizes": (w,), "val_acc": val_acc_cfg[-1]})
    print("-" * 50)

best_width = max(width_results, key=lambda r: r["val_acc"])["hidden_sizes"][0]
print("Best single-layer width: {}".format(best_width))

depth_configs = [(best_width,), (best_width, 128), (best_width, 128, 64)]
depth_results = []
for hs in depth_configs:
    print("Testing hidden_sizes={}".format(hs))
    torch.manual_seed(SEED)
    net_cfg = TuneNet(hidden_sizes=hs)
    _, _, val_acc_cfg = train(
        net_cfg, tune_train_loader, epochs=TUNE_EPOCHS, lr=BEST_LR, momentum=BEST_MOMENTUM,
        device=device, print_every=10000, val_loader=tune_val_loader
    )
    depth_results.append({"hidden_sizes": hs, "val_acc": val_acc_cfg[-1]})
    print("-" * 50)

best_architecture = max(depth_results, key=lambda r: r["val_acc"])
print("Best architecture: {}".format(best_architecture["hidden_sizes"]))
print("Best val accuracy: {:.4f}".format(best_architecture["val acc"]))
```

✓ 2m 12.9s

```

Epoch 1 finished | Avg Loss: 0.4026 | Train Acc: 0.8827 | Val Acc: 0.9445
Epoch 2 | Batch 0 | Loss: 0.2085
Test Accuracy: 0.9646
Epoch 2 finished | Avg Loss: 0.1323 | Train Acc: 0.9596 | Val Acc: 0.9646
Epoch 3 | Batch 0 | Loss: 0.0759
Test Accuracy: 0.9699
Epoch 3 finished | Avg Loss: 0.0858 | Train Acc: 0.9737 | Val Acc: 0.9699
Epoch 4 | Batch 0 | Loss: 0.1172
Test Accuracy: 0.9727
Epoch 4 finished | Avg Loss: 0.0606 | Train Acc: 0.9816 | Val Acc: 0.9727
-----
Testing hidden_sizes=(512, 128, 64)
Epoch 1 | Batch 0 | Loss: 2.2966
Test Accuracy: 0.9503
Epoch 1 finished | Avg Loss: 0.4929 | Train Acc: 0.8482 | Val Acc: 0.9503
Epoch 2 | Batch 0 | Loss: 0.1080
Test Accuracy: 0.9608
Epoch 2 finished | Avg Loss: 0.1285 | Train Acc: 0.9618 | Val Acc: 0.9608
Epoch 3 | Batch 0 | Loss: 0.1081
Test Accuracy: 0.9664
Epoch 3 finished | Avg Loss: 0.0850 | Train Acc: 0.9743 | Val Acc: 0.9664
Epoch 4 | Batch 0 | Loss: 0.1431
Test Accuracy: 0.9719
Epoch 4 finished | Avg Loss: 0.0622 | Train Acc: 0.9813 | Val Acc: 0.9719
-----
Best architecture: (512, 128)
Best val accuracy: 0.9727

```

Width sweep results: [{'hidden_sizes': (64,), 'val_acc': 0.9609}, {'hidden_sizes': (128,), 'val_acc': 0.9643}, {'hidden_sizes': (256,), 'val_acc': 0.9664}, {'hidden_sizes': (512,), 'val_acc': 0.9692}]

Depth sweep results: [{'hidden_sizes': (512,), 'val_acc': 0.9698}, {'hidden_sizes': (512, 128), 'val_acc': 0.9727}, {'hidden_sizes': (512, 128, 64), 'val_acc': 0.9719}]

Best architecture configuration: 2 hidden layers with neurons of sizes (521, 128) at an accuracy of 0.9727

And a final test, we trained the model with the optimal/best configurations after hyper-tuning:

```

torch.manual_seed(SEED)
final_net = TuneNet(hidden_sizes=best_architecture["hidden_sizes"])
train(final_net, train_loader, epochs=15, lr=BEST_LR, momentum=BEST_MOMENTUM, device=device)
final_test_acc = test(final_net, test_loader, device=device)
print("Final test accuracy with tuned config: {:.4f}".format(final_test_acc))

```

✓ 1m 24.8s

```

Epoch 1 | Batch 0 | Loss: 2.3116
Epoch 1 | Batch 200 | Loss: 1.0680
Epoch 1 | Batch 400 | Loss: 0.3851
Epoch 1 | Batch 600 | Loss: 0.3636
Epoch 1 | Batch 800 | Loss: 0.1297
Epoch 1 | Batch 1000 | Loss: 0.0824
Epoch 1 | Batch 1200 | Loss: 0.5954
Epoch 1 | Batch 1400 | Loss: 0.1465
Epoch 1 | Batch 1600 | Loss: 0.2176
Epoch 1 | Batch 1800 | Loss: 0.1045
Epoch 1 finished | Avg Loss: 0.3681 | Train Acc: 0.8908
Epoch 2 | Batch 0 | Loss: 0.1467
Epoch 2 | Batch 200 | Loss: 0.1913
Epoch 2 | Batch 400 | Loss: 0.1206
Epoch 2 | Batch 600 | Loss: 0.0804
Epoch 2 | Batch 800 | Loss: 0.0685
Epoch 2 | Batch 1000 | Loss: 0.2561
Epoch 2 | Batch 1200 | Loss: 0.0882
Epoch 2 | Batch 1400 | Loss: 0.1090
Epoch 2 | Batch 1600 | Loss: 0.1379
Epoch 2 | Batch 1800 | Loss: 0.0330
Epoch 2 finished | Avg Loss: 0.1189 | Train Acc: 0.9641
Epoch 3 | Batch 0 | Loss: 0.0081
Epoch 3 | Batch 200 | Loss: 0.6046
Epoch 3 | Batch 400 | Loss: 0.0171
...
Epoch 15 | Batch 1800 | Loss: 0.0003
Epoch 15 finished | Avg Loss: 0.0013 | Train Acc: 0.9999
Test Accuracy: 0.9844
Final test accuracy with tuned config: 0.9844

```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Task 9: Save your source code

Instructions:

- Save your python source code

Deliverable:

- Submit your Github link of your source code along with this document, which includes all screenshots via Blackboard.

- End of Assignment -

Grading Rubric

Criterion	Excellent	Good	Developing	Poor	Marks
Task 1: Install PyTorch	PyTorch installed successfully and PyTorch loaded correctly with clear structure	Installed and PyTorch loaded with minor issues	pyTorch exists but unclear or incomplete	Not completed / missing	0.5

Task 2: Download and Load the Dataset	Dataset loaded correctly with clear screenshot showing successful execution	Loaded correctly but screenshot is unclear	Dataset loaded with issues or incomplete evidence	Not loaded / missing screenshot	0.5
Task 3: Inspect the Dataset	All clear screenshots	Minor mistakes or missing small details	Several missing description or unclear screenshots	Not answered / incorrect / missing evidence	0.5
Task 4: Creating A Simple Network	A simple network correctly created, screenshot included	Minor reporting issues	Node used but missing result explanation or screenshot	Not done	2
Task 6: Training Function	Training function configured correctly with clear screenshots of output	Minor reporting issues	Training Function used but missing result explanation or screenshot	Not done	1
Task 7: Testing Network	Correct Testing network created, screenshot included.	Testing network correct but result description is weak	Testing network exists but unclear results description	Not done	1
Task 8: Tuning the parameters and model architecture	Tuning parameters and model architecture created, screenshot included, and results description explained accurately	Tuning Parameters and model architecture correct but results description is weak	Tuning Parameters and model architecture correct but incorrect results description	Not done	1.5
Task 9: Save your source code	Source code save successfully	Source code save successfully done but minor missing item	Source code save incomplete or file format incorrect	Not submitted / missing	0.5
Total					7.5